

```

---
## 1. Reverse String

### Problem Statement

Write a Python function `reverse_string(s: str)` that returns the reverse of a string **without using slicing**.

**Note:** Using slicing will receive zero marks.

**Example:**

```python
s = "Hello"
print(reverse_string(s))
Expected Output: "olleH"
```

### Solution

```python
test.py -r 'python test.py'

<template>
def reverse_string(s):
 <los>...</los>
 <sol>
 rev = ""
 for ch in s:
 rev = ch + rev
 return rev
 </sol>
</template>

import inspect
Automated check to ensure slicing is not used
source_code = inspect.getsource(reverse_string)
assert '[' not in source_code and ':' not in source_code, "0 marks: Slicing used!"

<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>

Public Test Cases

Input 1

```python
is_equal(reverse_string("Hello"), "olleH")
```

Output 1

...
'olleH'
...

Private Test Cases

Input 1

```python
is_equal(reverse_string("Hello World"), "dlroW olleH")
```

Output 1

...
'dlroW olleH'
...

Input 2

```python
is_equal(reverse_string(""), "")
```

Output 2

...
''
...

```

## ## 2. Count Vowels

### ### Problem Statement

Define a function `count\_vowels(s: str) -> int` that returns the number of vowels (a, e, i, o, u in both uppercase and lowercase) in a given string.

**Example:**

```
python
s = "Education"
print(count_vowels(s))
Expected Output: 5
```

### ### Solution

```
python
test.py -r 'python test.py'
```

```
<template>
def count_vowels(s: str) -> int:
 <!--...-->
 <sol>
 vowels = "aeiouAEIOU"
 count = 0
 for ch in s:
 if ch in vowels:
 count += 1
 return count
 </sol>
</template>

<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
...
```

### ### Public Test Cases

**Input 1**

```
python
is_equal(count_vowels("Education"), 5)
...
```

**Output 1**

```
...
5
...
```

### ### Private Test Cases

**Input 1**

```
python
is_equal(count_vowels("Rhythm"), 0)
...
```

**Output 1**

```
...
0
...
```

**Input 2**

```
python
is_equal(count_vowels(""), 0)
...
```

**Output 2**

```
...
0
...
```

### ## 3. Find Maximum

#### ### Problem Statement

Write a function `find_max(lst: list) -> int` that returns the maximum element of a list **without using the built-in `max()` function**.

**Note:** Using `max()` will receive zero marks.

**Example:**

```
'''python
lst = [4, 18, 2, 7]
print(find_max(lst))
Expected Output: 18
...'''
```

#### ### Solution

```
'''python
test.py -r 'python test.py'
```

```
<template>
def find_max(lst: list) -> int:
 <los>...</los>
 <sol>
 if not lst:
 raise ValueError("List cannot be empty")
 max_val = lst[0]
 for n in lst:
 if n > max_val:
 max_val = n
 return max_val
 </sol>
</template>
```

```
import inspect
Automated check to ensure max() is not used
source_code = inspect.getsource(find_max)
assert 'max(' not in source_code, "8 marks: Built-in max() function used!"
```

```
<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
...'''
```

#### ### Public Test Cases

**Input 1**

```
'''python
is_equal(find_max([4, 18, 2, 7]), 18)
...'''
```

**Output 1**

```
...
18
...'''
```

#### ### Private Test Cases

**Input 1**

```
'''python
is_equal(find_max([-5, -18, -3, -8]), -3)
...'''
```

**Output 1**

```
...
-3
...'''
```

**Input 2**

```
'''python
is_equal(find_max([42]), 42)
...'''
```

**Output 2**

```
...
42
...'''
```

#### ## 4. Replace Odd Numbers in NumPy Array

##### ### Problem Statement

Write a Python function 'replace\_odds(arr: np.ndarray) -> np.ndarray' that replaces all odd numbers in a NumPy array with '-1'.

**Example:**

```
'''python
import numpy as np
arr = np.arange(1, 11)
print(replace_odds(arr))
Expected Output: [-1 2 -1 4 -1 6 -1 8 -1 10]
'''
```

##### ### Solution

```
'''python
test.py -r 'python test.py'
```

```
import numpy as np
```

```
<template>
def replace_odds(arr: np.ndarray) -> np.ndarray:
 <los>...</los>
 <sol>
 arr = arr.copy()
 arr[arr % 2 != 0] = -1
 return arr
 </sol>
</template>
```

```
<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
'''
```

##### ### Public Test Cases

**Input 1**

```
'''python
is_equal(replace_odds(np.arange(1, 11)), np.array([-1, 2, -1, 4, -1, 6, -1, 8, -1, 10]))
'''
```

**Output 1**

```
...
array([-1, 2, -1, 4, -1, 6, -1, 8, -1, 10])
'''
```

##### ### Private Test Cases

**Input 1**

```
'''python
is_equal(replace_odds(np.array([0, 1, -3, 4, 5, -6])), np.array([0, -1, -1, 4, -1, -6]))
'''
```

**Output 1**

```
...
array([0, -1, -1, 4, -1, -6])
'''
```

**Input 2**

```
'''python
is_equal(replace_odds(np.array([11, 13, 15])), np.array([-1, -1, -1]))
'''
```

**Output 2**

```
...
array([-1, -1, -1])
'''
```

```

5. 2D Min-Max Normalization

Problem Statement

Write a function `two_d_norm(X)` that normalizes a 2D matrix `X` so all elements lie between 0 and 1 using global min-max normalization.

Formula: $X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$

Example:

```
python
import numpy as np
X = np.array([[1, 2], [3, 4]])
print(two_d_norm(X))
# Expected Output: [[0.    0.33]
#                  [0.67 1.   ]]

```


Solution


```
python
test.py -r 'python test.py'

import numpy as np

<template>
def two_d_norm(X):
    <los>...</los>
    <sol>
        X_min = X.min()
        X_max = X.max()
        X_norm = (X - X_min) / (X_max - X_min)
        return X_norm
    </sol>
</template>

<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>

```


Public Test Cases

Input 1*

```
python
is_equal(two_d_norm(np.array([[1, 2], [3, 4]])),
         np.array([[0., 0.3333333], [0.6666667, 1.])))

```

Output 1*

```

array([[0.    , 0.3333333],
       [0.6666667, 1.    ]])

```


Private Test Cases

Input 1*

```
python
X1 = np.array([[10, 20, 30], [5, 15, 25], [0, 50, 100]])
is_equal(two_d_norm(X1),
         np.array([[0.1, 0.2, 0.3], [0.05, 0.15, 0.25], [0., 0.5, 1.])))

```

Output 1*

```

array([[0.1 , 0.2 , 0.3 ],
       [0.05, 0.15, 0.25],
       [0.  , 0.5 , 1.   ]])

```

Input 2*

```
python
X2 = np.array([[-1000, 0, 1000], [5000, -500, 2000], [10000, 500, -2000]])
result = two_d_norm(X2)
is_equal(result.min(), 0.0)
is_equal(result.max(), 1.0)

```

Output 2*

```

All values normalized between 0 and 1

```


```

## ## 6. Shopping Cart

### ### Problem Statement

Implement a 'ShoppingCart' class to manage items with the following specifications:

- Store items as tuples of '(name, price)'
- Method 'add\_item(name: str, price: float)' - adds an item to cart
- Method 'remove\_item(name: str) -> bool' - removes first occurrence of item, returns 'True' if found
- Method 'calculate\_total()' -> float' - returns sum of all item prices

**\*\*Example:\*\***

```
'''python
cart = ShoppingCart()
cart.add_item("Papaya", 180)
cart.add_item("Guava", 200)
print(cart.calculate_total())
Expected Output: 380
'''
```

### ### Solution

```
'''python
test.py -r 'python test.py'
```

```
<template>
class ShoppingCart:
 <los>...</los>
 <sol>
 def __init__(self):
 self.items = []

 def add_item(self, name: str, price: float) -> None:
 self.items.append((name, price))

 def remove_item(self, name: str) -> bool:
 for item in self.items:
 if item[0] == name:
 self.items.remove(item)
 return True
 return False

 def calculate_total(self) -> float:
 return sum(price for _, price in self.items)
 </sol>
</template>
```

```
<suffix_invisible>
{% include "class_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
...
```

### ### Public Test Cases

**\*\*Input 1\*\***

```
'''python
cart = ShoppingCart()
cart.add_item("Papaya", 180)
cart.add_item("Guava", 200)
is_equal(cart.calculate_total(), 380)
'''
```

**\*\*Output 1\*\***

```
...
380
...
```

### ### Private Test Cases

**\*\*Input 1\*\***

```
'''python
cart2 = ShoppingCart()
cart2.add_item("Apple", 50)
cart2.add_item("Apple", 70)
cart2.remove_item("Apple")
is_equal(cart2.calculate_total(), 70)
'''
```

**\*\*Output 1\*\***

```
...
70
...
```

**\*\*Input 2\*\***

```
'''python
cart3 = ShoppingCart()
is_equal(cart3.calculate_total(), 0)
'''
```

**\*\*Output 2\*\***

```
...
0
...
```

## ## 7. Apply Discount

### ### Problem Statement

Write `apply_discount(order_list)` which applies discounts using a **lambda function** based on customer type

- **Prime**: 28% discount (multiply by 0.8)
- **Regular**: 18% discount if amount > 1800 (multiply by 0.9), else no discount
- **Guest**: No discount

Input format: List of tuples `[(customer_type, amount), ...]`

**Example:**

```
'''python
orders = [("regular", 1288.0), ("prime", 800.0), ("guest", 1500.0)]
print(apply_discount(orders))
Expected Output: [1888.0, 640.0, 1500.0]
'''
```

### ### Solution

```
'''python
test.py -r 'python test.py'
```

```
<template>
def apply_discount(order_list):
 <los>...</los>
 <sol>
 discount = lambda ctype, amt: (
 amt * 0.8 if ctype == "prime" else
 amt * 0.9 if ctype == "regular" and amt > 1800 else
 amt
)
 return [discount(c, a) for c, a in order_list]
 </sol>
</template>
```

```
import inspect
Automated check to ensure lambda is used
source_code = inspect.getsource(apply_discount)
assert 'lambda' in source_code, "0 marks: Lambda function not used!"
```

```
<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
'''
```

### ### Public Test Cases

**Input 1**

```
'''python
orders = [("regular", 1288.0), ("prime", 800.0), ("guest", 1500.0)]
is_equal(apply_discount(orders), [1888.0, 640.0, 1500.0])
'''
```

**Output 1**

```
...
[1888.0, 640.0, 1500.0]
'''
```

### ### Private Test Cases

**Input 1**

```
'''python
orders2 = [("prime", 500.0), ("prime", 1500.0), ("regular", 2800.0),
 ("regular", 800.0), ("guest", 3000.0)]
is_equal(apply_discount(orders2), [400.0, 1200.0, 1800.0, 800.0, 3000.0])
'''
```

**Output 1**

```
...
[400.0, 1200.0, 1800.0, 800.0, 3000.0]
'''
```

**Input 2**

```
'''python
orders3 = [("prime", 100000.0), ("regular", 1001.0), ("guest", 99999.0),
 ("regular", 999.0), ("prime", 50.0)]
is_equal(apply_discount(orders3), [80000.0, 999.0, 99999.0, 999.0, 50.0])
'''
```

**Output 2**

```
...
[80000.0, 999.0, 99999.0, 999.0, 50.0]
'''
```

Write `group_sales_total(data_dict, month_filter, group_var, group_value)` that:

1. Converts a dictionary to a Pandas DataFrame
2. Filters rows by the specified month
3. Groups by the `group_var` column
4. Sums the "Sales" column
5. Returns the total for the specified `group_value` (or 0 if not found)

**Example:**

```
'''python
sales_data = {
 "Region": ["North", "South", "East", "West", "North", "East", "South", "West"],
 "Product": ["A", "A", "B", "B", "C", "C", "A", "B"],
 "Sales": [1280, 880, 680, 980, 1500, 700, 650, 1100],
 "Month": ["Jan", "Jan", "Feb", "Feb", "Jan", "Feb", "Feb", "Jan"]
}
print(group_sales_total(sales_data, "Jan", "Region", "North"))
Expected Output: 1280
'''
```

**Solution**

```
'''python
test.py -r 'python test.py'

import pandas as pd

<template>
def group_sales_total(data_dict, month_filter, group_var, group_value):
 <los>...</los>
 <sol>
 df = pd.DataFrame(data_dict)
 df_filtered = df[df["Month"] == month_filter]
 grouped = df_filtered.groupby(group_var)["Sales"].sum()
 return grouped.get(group_value, 0)
 </sol>
</template>

<suffix_invisible>
{% include "function_type_and_modify_check_suffix.py.jinja" %}
</suffix_invisible>
'''
```

**Public Test Cases**

**Input 1**

```
'''python
sales_data = {
 "Region": ["North", "South", "East", "West", "North", "East", "South", "West"],
 "Product": ["A", "A", "B", "B", "C", "C", "A", "B"],
 "Sales": [1280, 880, 680, 980, 1500, 700, 650, 1100],
 "Month": ["Jan", "Jan", "Feb", "Feb", "Jan", "Feb", "Feb", "Jan"]
}
is_equal(group_sales_total(sales_data, "Jan", "Region", "North"), 1280)
'''
```

**Output 1**

```
...
1280
...
```

**Private Test Cases**

**Input 1**

```
'''python
is_equal(group_sales_total(sales_data, "Jan", "Region", "West"), 1100)
'''
```

**Output 1**

```
...
1100
...
```

**Input 2**

```
'''python
is_equal(group_sales_total(sales_data, "Feb", "Product", "C"), 780)
'''
```

**Output 2**

```
...
780
...
```

**Input 3**

```
'''python
is_equal(group_sales_total(sales_data, "Jan", "Region", "Central"), 0)
'''
```

**Output 3**

```
...
0
```



## ## Summary

This collection covers 8 essential Python programming concepts formatted in the template code theme:

- **String Manipulation** (Problems 1, 2)
- **List Operations** (Problem 3)
- **NumPy Array Processing** (Problems 4, 5)
- **Object-Oriented Programming** (Problem 6)
- **Lambda Functions** (Problem 7)
- **Pandas Data Analysis** (Problem 8)

Each problem includes automated checks, comprehensive test cases, and follows the standardized template format.